

ESTATE MANAGER WEB APP

Product Requirements Document (PRD)

Prototype & MVP 1

Product Name: Estate Manager Web App

Platform: Web Application

Architecture: MERN Stack

Version: Prototype / MVP 1

1. PRODUCT OVERVIEW

The Estate Manager Web App is a centralized property and estate management platform designed to connect residents/homeowners, estate facility management teams, security personnel and administrators through one digital ecosystem.

The platform will provide:

- Resident/property management
- Estate administration
- Access-code management
- Visitor verification
- Security monitoring
- Invoice and payment management
- Complaints and service requests
- Estate-wide communication
- Community communication
- Facility booking
- Emergency/SOS alerts
- Notifications (PWA)
- Administrative reporting
- Audit and activity tracking

The system should be designed as a multi-estate platform so that one installation can manage multiple residential estates.

2. PRODUCT OBJECTIVES

The primary objectives are to:

1. Digitize estate management operations.

2. Give residents a single platform for estate-related services.
3. Improve security and visitor access management.
4. Improve visibility of invoices, payments and outstanding bills.
5. Improve communication between residents and estate management.
6. Create a structured complaint/request management process.
7. Provide estate administrators with centralized operational dashboards.
8. Enable facility booking.
9. Provide emergency/SOS communication.
10. Create a secure and auditable administrative system.
11. Support multiple estates and multiple properties per resident.
12. Integrate with an external onboarding platform through APIs.

3. USER CATEGORIES

The system will support five primary roles.

3.1 Resident/Homeowner

A person who lives in, owns, rents or is otherwise authorized to occupy a property within an estate.

Residents can:

- Manage their profile
- View their properties
- View dependants
- Generate access codes
- Manage their access codes where permitted
- View invoices
- Make payments
- View/download payment history
- Submit complaints
- Track complaints
- Receive estate messages
- Participate in community communication
- Book estate facilities
- Trigger SOS alerts
- Receive notifications

4. SECURITY GUARD

A security guard is responsible for verifying access codes and visitor evidence at an estate gate/security point.

Guards can:

- Sign in
- Enter/scan access codes
- Verify access codes
- Capture visitor images
- Capture vehicle images where required
- Approve/reject access
- View permitted verification history
- View their own profile

Guards must NOT have access to:

- Resident financial information
- Invoice details
- Payment history
- Resident complaints
- Private resident communication
- Administrative functions
- Unrelated estate information

Guard access must be restricted to the estate(s) they are assigned to.

5. ADMIN

An Admin is an estate/facility management staff member responsible for day-to-day estate operations.

Admins may have access to:

- Residents
- Properties
- Invoices
- Payments
- Complaints
- Messages
- Access codes
- Verification history
- Facilities
- Bookings
- Reports

Admin access must be restricted by estate assignment.

For example:

Admin A:

- Estate A ✓
- Estate B ✓
- Estate C ✕

Admin A must not be able to access Estate C simply by changing an `estateId` in the browser or API request.

6. SUPER ADMIN

The Super Admin is the highest-level administrator.

Super Admin can:

- Manage all estates
- Manage users
- Manage administrators
- Assign admins to estates
- Manage permissions
- Manage properties
- Manage residents
- Manage invoices
- View payments
- Manage access codes
- Manage visitor verification records
- Manage complaints
- Broadcast messages
- Manage facilities
- Manage bookings
- Manage SOS alerts
- View reports
- View audit logs
- Disable/reactivate resident access-code generation
- Manage system settings

7. AUDITOR

The Auditor is a read-only role designed for oversight and accountability.

Auditors can view approved information including:

- Residents
- Properties
- Invoices
- Payments
- Access-code history
- Verification records
- Complaints
- Audit logs
- Reports

Auditors cannot:

- Create invoices
- Edit invoices
- Delete records
- Generate access codes
- Deactivate access codes
- Disable resident access-code generation
- Change permissions
- Modify users
- Broadcast messages

8. RBAC AND AUTHORIZATION MODEL

The application must implement:

RBAC + Permission-Based Access Control + Resource-Level Authorization

Authentication answers:

Who are you?

RBAC answers:

What role do you have?

Permissions answer:

What actions can your role perform?

Resource authorization answers:

Which estate/property/user/resource can you perform that action on?

The final authorization model is:

AUTHENTICATED USER



ROLE



PERMISSION



ESTATE SCOPE



RESOURCE OWNERSHIP



ACCESS GRANTED / DENIED

Frontend authorization must NEVER be considered sufficient.

Every protected backend endpoint must independently enforce authorization.

9. ROLES AND PERMISSIONS

RESIDENT

Typical permissions:

profile.view
profile.edit
property.view
invoice.view
payment.create
payment.view
code.create
code.view
code.deactivate
code.extend
complaint.create
complaint.view
message.view
community.view
community.post
facility.view
facility.book
booking.view
sos.create
notification.view

SECURITY GUARD

profile.view
verification.create
verification.view
evidence.upload

ADMIN

residents.view
residents.edit
properties.view
properties.edit
invoices.view

invoices.create
invoices.edit
invoices.cancel
payments.view
codes.view
codes.deactivate
codes.extend
verification.view
complaints.view
complaints.manage
messages.view
messages.broadcast
facilities.manage
bookings.manage
sos.manage
reports.view

SUPER ADMIN

ALL PERMISSIONS

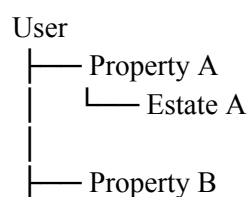
AUDITOR

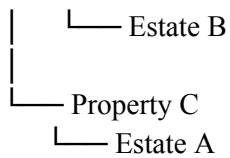
residents.view
properties.view
invoices.view
payments.view
codes.view
verification.view
complaints.view
reports.view
audit_logs.view

10. ESTATE AND PROPERTY DATA MODEL

A user must be able to have multiple properties.

Example:

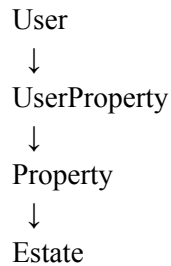




Do NOT design the database around:

one user = one property

Instead use a relationship model:



This allows the same resident/homeowner to appear against multiple properties.

11. PROPERTY TYPES

The system should support:

- Land
- Apartment
- Terrace
- Detached house
- Semi-detached house
- Penthouse
- Other configurable types

Property description examples:

- 1 Bedroom
- 2 Bedroom
- 3 Bedroom
- 2 Bedroom Penthouse
- Block A Apartment 223
- Plot 01
- Court A - A223

12. PROPERTY IDENTIFICATION

The system should support structured property references.

Example:

Estate: Mansfield Estate
Property Type: Apartment
Court: A
Block: 02
Floor: 02
Apartment: A223

Display can be simplified for users:

Court A - A223

The full structured information should remain available in the database.

13. DEPENDANTS

A resident may have dependants.

Example:

```
{  
  "firstName": "Wisdom",  
  "lastName": "Nnadi",  
  "email": "example@example.com",  
  "phone": "07000000000",  
  "gender": "Male"  
}
```

Dependants should be associated with the resident.

Admin dashboard should NOT display all dependant information by default.

Recommended display:

Resident
Paul Nnadi

Dependants
2

The admin can open the resident profile to view authorized dependant details.

Sensitive dependant information should be protected by permission.

14. EXTERNAL ONBOARDING INTEGRATION

The Estate Manager Web App will connect to a separate onboarding platform.

The onboarding platform is responsible for collecting initial resident/property information.

Example onboarding payload:

```
{
  "firstName": "Paul",
  "lastName": "Nnadi",
  "email": "example@example.com",
  "phone": "07000000000",
  "gender": "male",
  "maritalStatus": "Single",
  "propertyInfo": {
    "propertyType": 0,
    "address": "Courtu BlockBa",
    "houseType": "2 Bedroom"
  },
  "estate": {
    "name": "Mansfield Estate",
    "address": "Surulere, Lagos State"
  },
  "dependants": [
    {
      "firstName": "Wisdom",
      "lastName": "Nnadi",
      "email": "example@example.com",
      "phone": "07000000000",
      "gender": "Male"
    }
  ]
}
```

The Estate Manager Web App should support:

PUSH
GET
PULL
SYNC

API integration.

The frontend must NOT connect directly to the external onboarding database.

All integration must happen through the backend.

15. ONBOARDING FLOW

EXTERNAL ONBOARDING PLATFORM



API PUSH



ESTATE MANAGER BACKEND



VALIDATE DATA



CREATE/UPDATE



USER

PROPERTY

ESTATE

USER-PROPERTY

DEPENDANTS



RESIDENT CAN SIGN IN

The integration must be idempotent.

If the same onboarding record is sent twice, the application must not create duplicate residents or properties.

Store the external onboarding identifier.

16. RESIDENT DASHBOARD

The resident dashboard should contain:

Summary cards

- Outstanding bills
- Amount paid
- Active access codes
- Open complaints
- Upcoming bookings
- Notifications

Main sections

My Properties

Outstanding Bills

Access Codes
Recent Payments
Complaints
Announcements
Upcoming Bookings
Notifications

Dashboard data should update automatically when the resident signs in.

17. RESIDENT PROFILE

Resident can view:

- Full name
- Email
- Phone
- Gender
- Marital status
- Profile image
- Properties
- Dependants

Resident should only be able to edit fields permitted by the system.

Estate assignment and property ownership information should not be freely editable by the resident.

18. INVOICE MANAGEMENT

Invoices should contain:

invoiceNumber
userId
propertyId
estateId
description
amount
amountPaid
balance
dueDate
status
createdBy
createdAt
updatedAt

Statuses:

DRAFT
PENDING
PARTIALLY_PAID
PAID
OVERDUE
CANCELLED

19. BILL CREATION

Admin/Super Admin can:

- Create individual invoice
- Create bulk invoice
- Push invoice to residents within an estate
- Edit invoice
- Cancel invoice
- Search invoices
- Filter invoices
- Sort invoices

Bulk billing must allow:

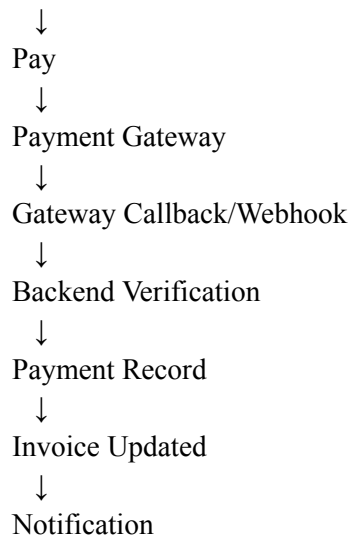
Select Estate
↓
Select Bill Type
↓
Enter Amount
↓
Enter Description
↓
Set Due Date
↓
Review
↓
Create/Push

20. PAYMENT SYSTEM

The application should integrate with a payment gateway.

Payment flow:

Resident
↓
Select Invoice



The frontend must NEVER directly mark an invoice as paid.

Payment success must be determined through server-side verification.

21. PAYMENT HISTORY

Residents can view:

- Payment date
- Invoice number
- Amount
- Payment reference
- Payment status
- Description

Residents should be able to download payment records where applicable.

Admins can search/filter payment history by:

- Estate
- Resident
- Property
- Invoice
- Date
- Status

22. REAL-TIME PAYMENT UPDATE

When payment is successfully verified:

Payment

↓
Invoice balance updated
↓
Invoice status updated
↓
Notification generated
↓
Dashboard refresh/update

Duplicate payment webhooks must not create duplicate payment records.

23. ACCESS-CODE MANAGEMENT

Residents can generate access codes for authorized entry/exit/visitor use.

Access code fields:

codeHash
residentId
propertyId
estateId
type
status
validFrom
expiresAt
validationMethods
maxUses
usageCount
createdAt
updatedAt

Statuses:

ACTIVE
EXPIRED
DEACTIVATED
USED

24. ACCESS-CODE FUNCTIONS

Resident:

- Generate code
- View active codes
- View expired codes
- Deactivate code

- Extend code expiry

Admin:

- View codes within assigned estate
- Deactivate codes
- Extend codes where permitted
- View code activity

Super Admin:

- Full code management
- Disable resident code-generation privilege
- Reactivate resident code-generation privilege
- View access-code audit history

25. SUPER ADMIN — CODE GENERATION RESTRICTION

This is a critical business rule.

A resident's entire account must NOT be suspended simply because they owe money.

Instead, Super Admin can disable the resident's:

ACCESS-CODE GENERATION PRIVILEGE

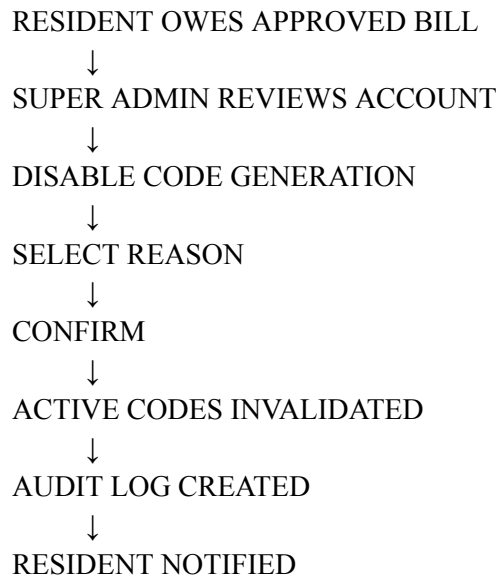
The resident may continue using:

- Dashboard
- Invoices
- Payments
- Complaints
- Messages
- Community
- Facilities
- Notifications

However, they cannot:

- Generate new access codes
- Extend access codes
- Reactivate access codes

26. CODE GENERATION RESTRICTION FLOW



Example reasons:

Service charge arrears
Utility arrears
Estate levy arrears
Other approved estate charge
Security-related restriction
Administrative restriction
Other

A custom reason should be supported.

27. ACCESS-CONTROL STATUS

Resident account should have:

accessControlStatus

Values:

ENABLED
DISABLED

Additional fields:

accessControlDisabledReason
accessControlDisabledAt

accessControlDisabledBy
accessControlReenabledAt
accessControlReenabledBy

When disabled:

POST /api/codes

must return:

403 ACCESS_CONTROL_DISABLED

The frontend must display:

Access code generation is currently disabled for your account. Please contact the Estate Office.

28. IMPORTANT DISTINCTION

There are two different actions:

Individual code deactivation

Deactivate Code #ABC123

Only that particular code is disabled.

Resident access-code restriction

Disable Access-Code Generation

This affects the resident's ability to generate/manage codes.

These must NOT be treated as the same database state.

29. ACCESS-CODE SECURITY

Codes must:

- Be cryptographically generated
- Be unpredictable
- Have expiry
- Have usage limits where applicable
- Be scoped to estate/property
- Be securely stored
- Be independently validated by the backend

Never trust:

estateId
propertyId
residentId

provided by the client without server-side verification.

30. SECURITY GUARD VERIFICATION

Guard verification flow:



31. VALIDATION METHODS

The following validation methods will be supported:

visitor_image
vehicle_image

Resident ID

Resident ID must NOT be used as a visitor validation method.

It should not appear in:

- UI
- API schemas
- validation enums
- database configuration
- guard interface
- verification logic

32. VISITOR IMAGE CAPTURE

If:

`visitor_image`

is selected, capturing the visitor image is mandatory.

Guard flow:

Take Photo



Preview



Retake / Confirm



Upload



Verify

Verification must fail if the required image is missing.

Return:

400 REQUIRED_VISITOR_IMAGE

33. VEHICLE IMAGE

If:

`vehicle_image`

is selected, vehicle image capture is mandatory.

The system should support:

visitor_image
+
vehicle_image

as a combined validation method.

34. VERIFICATION LOG

Every verification should create a permanent log.

VerificationLog

Fields:

accessCodeId
residentId
propertyId
estateId
guardId
verificationType
result
visitorImage
vehicleImage
notes
verifiedAt
ipAddress
deviceId

Results:

APPROVED
REJECTED

Historical verification records cannot be deleted or edited by guards.

35. VISITOR IMAGE STORAGE

Do not store large image binaries directly inside MongoDB.

Use a storage abstraction:

storageService.upload()
storageService.delete()
storageService.getUrl()

Store:

storageKey
reference
mimeType
fileSize
uploadedAt

Validate:

- MIME type
- File size
- Image integrity
- Authentication
- Estate/resource authorization

Never trust the uploaded filename.

36. RESIDENT VERIFICATION NOTIFICATION

After verification, notify the resident.

Example:

Your visitor access code was verified at the estate gate.

Notification should include:

- Date/time
- Result
- Estate
- Access-code type
- Gate/location where configured

37. COMPLAINT MANAGEMENT

Residents can:

- Create complaint
- Report issue
- Submit request
- Upload evidence
- View complaint history
- Respond to updates

Complaint statuses:

OPEN
ASSIGNED
IN_TREATMENT
RESOLVED
CLOSED

Admin can:

- View
- Assign
- Treat
- Update
- Respond
- Resolve
- Close

38. COMPLAINT DASHBOARD

Admin complaint dashboard should have:

OPEN
ASSIGNED
IN TREATMENT
RESOLVED
CLOSED

Use color-coded status indicators.

Each complaint should contain:

Ticket Number
Resident
Property
Estate
Category
Subject
Description
Assigned Staff
Status
Created Date
Updated Date

39. TWO-WAY FM COMMUNICATION

The platform should provide communication between residents and estate management.

Resident:

Send request
Report issue
Make complaint
Receive response

FM team:

Respond
Send updates
Send reminders
Send announcements

40. ESTATE MESSAGE BOARD

Admin can broadcast messages to:

- One estate
- Multiple estates
- Selected residents

Message types:

GENERAL
ANNOUNCEMENT
INVOICE_REMINDER
MAINTENANCE
SECURITY
ESTATE_UPDATE

Message history should be retained.

Residents can:

- View messages
- Mark messages as read
- View message history

41. COMMUNITY

The platform should support intra-estate communication.

MVP functionality:

- Estate community feed

- General discussion
- Estate announcements
- Direct messaging
- Group messaging

Community permissions must prevent unauthorized access across estates.

42. FACILITY BOOKING

Residents should be able to book estate facilities.

Examples:

- Swimming pool
- Tennis court
- Gym
- Football pitch
- Event space
- Other estate facilities

Facility fields:

name
description
estateId
location
capacity
operatingHours
bookingRules
status

43. BOOKING

Booking fields:

facilityId
estateId
userId
propertyId
date
startTime
endTime
status
notes

Statuses:

PENDING
CONFIRMED
CANCELLED
COMPLETED

The backend must prevent overlapping bookings.

44. SOS / EMERGENCY

Residents can trigger an SOS alert.

Options:

Alert FM Team
Alert Estate
View Local First Responder Contacts

SOS fields:

userId
propertyId
estateId
alertType
status
createdAt
resolvedAt
respondersNotified

Statuses:

ACTIVE
ACKNOWLEDGED
RESOLVED
CANCELLED

Admin dashboard must show active alerts.

45. NOTIFICATIONS

Notification types:

PAYMENT_SUCCESS
INVOICE_CREATED
INVOICE_REMINDER
ACCESS_VERIFICATION
ACCESS_CONTROL_DISABLED

ACCESS_CONTROL_REENABLED
COMPLAINT_UPDATE
ESTATE_MESSAGE
BOOKING_UPDATE
SOS
SYSTEM

Residents should have:

- Notification center
- Unread count
- Read/unread state
- Notification timestamps

46. ADMIN DASHBOARD

Admin dashboard should contain:

Total Residents
Total Properties
Outstanding Bills
Amount Collected
Open Complaints
Overdue Invoices
Active Access Codes
Today's Verifications
Active SOS Alerts

Charts/reports can be introduced where useful.

47. RESIDENTS DASHBOARD

Resident management table:

Full Name
Phone Number
Email
Estate Name
Estate Location
Property Type
Property Description
Property Address
Relationship
Number of Units
Status

Date Added

Features:

- Search
- Sort
- Filter
- Pagination
- Newest to oldest
- Oldest to newest
- Ascending
- Descending

48. MULTIPLE PROPERTY HANDLING

If one resident has:

Property A → Apartment

Property B → Land

Property C → Terrace

each property relationship should be independently represented.

The admin dashboard can display:

Paul Nnadi | Apartment | Estate A

Paul Nnadi | Land | Estate B

Paul Nnadi | Terrace | Estate A

This is preferred for MVP because it makes property-level billing, access control and reporting easier.

49. ADMIN ESTATE SCOPING

Every admin account should have estate assignments.

Example:

Admin:

Jane

Assigned Estates:

Mansfield Estate

Fairfield Apartments

Jane can only access data associated with those estates.

Changing:

?estateId=another-estate

in the browser must not bypass authorization.

Backend must independently determine permitted estate scope.

50. AUDIT LOGGING

Sensitive operations must create audit records.

Audit fields:

actorId
actorRole
action
resourceType
resourceId
previousValue
newValue
reason
ipAddress
userAgent
timestamp

Log at minimum:

- Login
- Logout
- User creation
- User update
- Property changes
- Estate assignment
- Invoice creation
- Invoice modification
- Invoice cancellation
- Payment status changes
- Access-code creation
- Access-code deactivation
- Access-code extension
- Access-code restriction
- Access-code restriction removal
- Bulk code invalidation
- Complaint status changes
- Message broadcasts
- Permission changes

- SOS resolution

Never log:

- Passwords
- JWT secrets
- Raw authentication tokens
- Raw access codes

51. AUTHENTICATION SECURITY

The system must use short-lived JWT access tokens.

Default:

JWT_EXPIRES_IN=30m

JWT should contain only required claims.

Do not put sensitive resident information inside JWT payloads.

52. HTTPONLY AUTHENTICATION COOKIE

Where cookie-based authentication is used:

HttpOnly = true

Secure = true in production

SameSite = appropriate restrictive setting

Authentication cookies must not be readable by normal frontend JavaScript.

Where cookies are used, implement an appropriate CSRF protection strategy.

53. CONTENT SECURITY POLICY

CSP must be implemented.

Add CSP configuration to:

client/index.html

The production application should preferably also send CSP as an HTTP response header.

Starting policy should restrict:

default-src
script-src
style-src
img-src
font-src
connect-src
object-src
frame-ancestors
base-uri
form-action

The policy must be adjusted only for legitimate API, payment and storage domains.

54. OTHER SECURITY REQUIREMENTS

Implement:

- Helmet
- CORS allowlisting
- Rate limiting
- Input validation
- Request-size limits
- Secure password hashing
- Secure file uploads
- Centralized error handling
- MongoDB query protection
- Audit logs
- Resource-level authorization
- Secure payment webhook validation
- Secure access-code generation

55. TECHNOLOGY STACK

Frontend

React
Vite
React Router
Tailwind CSS
Axios
TanStack Query
React Hook Form
Zod

Lucide React

Backend

Node.js

Express.js

MongoDB

Mongoose

JWT

bcrypt/Argon2

Helmet

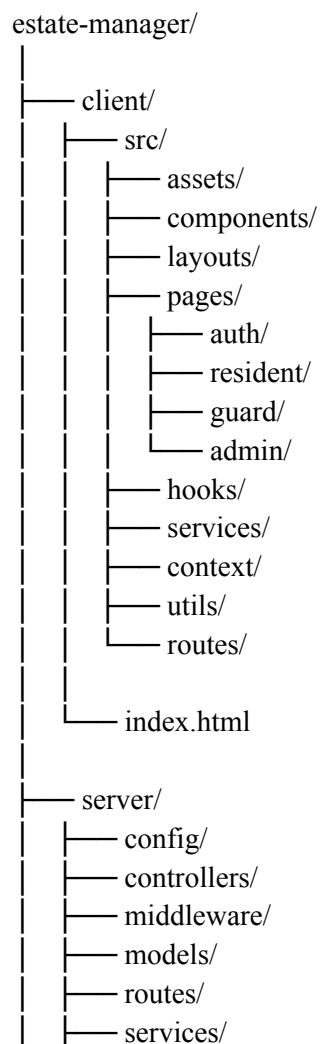
CORS

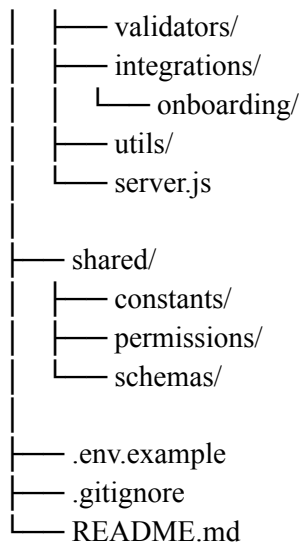
express-rate-limit

Zod

56. PROJECT STRUCTURE

Recommended structure:





57. CORE DATABASE MODELS

Minimum models:

User
Estate
Property
UserProperty
Dependant
EstateAdminAssignment
Invoice
Payment
AccessCode
VerificationLog
Complaint
Message
Notification
Facility
Booking
SOSAlert
AuditLog

58. API STRUCTURE

Recommended API:

/api/auth
/api/users
/api/estates
/api/properties

/api/dependants
/api/invoices
/api/payments
/api/codes
/api/verifications
/api/complaints
/api/messages
/api/notifications
/api/facilities
/api/bookings
/api/sos
/api/audit-logs
/api/integrations/onboarding

59. MVP 1 PRIORITY

P0 — Mandatory

Authentication
RBAC
Estate-level authorization
Onboarding integration
Residents
Properties
Invoices
Payments
Access codes
Super Admin code-generation restriction
Guard verification
Visitor image capture
Complaints
Notifications
Admin dashboard
Audit logs
Security controls

P1 — MVP Enhancement

Facility booking
Estate messaging
SOS
Vehicle image verification
Basic community
Direct messaging

P2 — Post-MVP

Advanced analytics
Advanced community features
Advanced reporting
Mobile application
Biometric integrations
Advanced gate hardware integration
Automated debt-management rules
Advanced facility scheduling

60. SUCCESS CRITERIA

The MVP is considered successful when:

Resident

A resident can:

Sign in
↓
View profile
↓
View properties
↓
View invoice
↓
Make payment
↓
Receive payment confirmation
↓
Generate access code
↓
Receive visitor verification notification
↓
Submit complaint
↓
Track complaint
↓
Book facility

Guard

A guard can:

Sign in
↓

Enter code
↓
Validate code
↓
Capture visitor image
↓
Capture vehicle image if required
↓
Approve/reject
↓
Create verification record

Admin

An admin can:

Sign in
↓
Access assigned estate
↓
View residents
↓
View properties
↓
Create invoices
↓
View payments
↓
Manage complaints
↓
Broadcast messages
↓
View access codes
↓
View verification history

Super Admin

A Super Admin can:

Manage all estates
↓
Manage staff
↓
Manage permissions
↓
View all residents
↓

Manage invoices



View payments



Manage access codes



Disable resident code generation



Reactivate resident code generation



View audit logs

61. CRITICAL SECURITY TEST CASES

The application must pass these tests:

Authentication

- Invalid JWT rejected
- Expired JWT rejected
- Invalid credentials rejected
- Brute-force attempts rate limited

Authorization

- Resident cannot access another resident's data
- Resident cannot modify another resident's property
- Resident cannot access admin APIs
- Guard cannot access invoices
- Guard cannot access resident financial information
- Auditor cannot modify resources
- Admin cannot access unassigned estate
- Admin cannot escalate privileges
- Only Super Admin can disable resident code generation

Access Codes

- Expired code rejected
- Deactivated code rejected
- Wrong-estate code rejected
- Wrong-property code rejected
- Invalid code rejected
- Disabled resident cannot generate a new code
- Disabled resident cannot extend a code
- Active codes are invalidated according to restriction policy
- Visitor image required when configured

- Vehicle image required when configured

Payments

- Invalid payment webhook rejected
- Duplicate webhook does not duplicate payment
- Wrong amount rejected
- Frontend cannot mark invoice as paid

File Uploads

- Unauthorized upload rejected
- Invalid MIME type rejected
- Oversized file rejected
- Unsafe filename rejected
- Unauthorized resource access rejected

62. IMPORTANT BUSINESS RULES

The following rules are mandatory:

Rule 1

A resident may have multiple properties.

Rule 2

Every property belongs to an estate.

Rule 3

Every property relationship must be independently authorized.

Rule 4

An Admin can only access assigned estates.

Rule 5

Super Admin has global administrative access.

Rule 6

Auditor is read-only.

Rule 7

Guard access is limited to security verification.

Rule 8

Resident ID is NOT a visitor validation method.

Rule 9

Visitor image capture is mandatory when configured.

Rule 10

Vehicle image capture is mandatory when configured.

Rule 11

Super Admin can disable code generation for residents who owe approved charges.

Rule 12

Disabling code generation does not suspend the entire resident account.

Rule 13

When code generation is disabled, the backend must enforce the restriction.

Rule 14

Sensitive administrative actions must be audited.

Rule 15

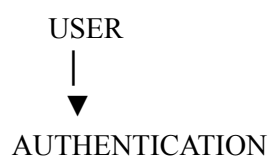
Payment success must be verified server-side.

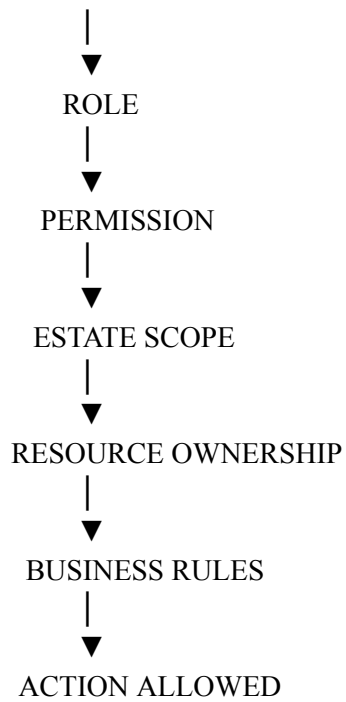
Rule 16

Frontend security controls must never replace backend authorization.

63. FINAL ARCHITECTURE PRINCIPLE

The Estate Manager Web App should be designed around the following security model:





For example:

Can this person generate an access code?

